

Chapter 21. Troubleshooting Applications

When operating an application in a production Kubernetes cluster, failures are almost inevitable. It's your responsibility as a Kubernetes administrator to be able to troubleshoot issues for the Kubernetes objects designed and deployed by an application developer.

In this chapter, we'll look at troubleshooting strategies that can help with identifying the root cause of an issue so that you can take action and correct the failure appropriately. The strategies discussed here start with the high-level perspective of a Kubernetes object and then drill into more detail as needed.

COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objectives:

- Manage and evaluate container output streams
 - Troubleshoot services and networking
 - Monitor cluster and application resource usage
-

Troubleshooting Pods

In most cases, creating a Pod is no issue. You simply emit the `run`, `create`, or `apply` commands to instantiate the Pod. If the YAML manifest is formed properly, Kubernetes accepts your request, so the assumption is that everything works as expected. To verify the correct behavior, the first thing you'll want to do is to check the Pod's high-level runtime information. The operation could involve other Kubernetes objects like a Deployment responsible for rolling out multiple replicas of a Pod.

Retrieving High-Level Information

To retrieve the information, run either the `kubectl get pods` command for just the Pods running in the namespace or the `kubectl get all` command to retrieve the most prominent object types in the namespace (which includes Deployments). You will want to look at the columns `READY`, `STATUS`, and `RESTARTS`. In the optimal case, the number of

ready containers matches the number of containers you defined in the `spec`. For a single-container Pod, the `READY` column would say `1/1`.

The status should say `Running` to indicate that the Pod entered the proper lifecycle state. Be aware that it's totally possible that a Pod renders a `Running` state, but the application isn't actually working properly. If the number of restarts is greater than 0, then you might want to check the logic of the liveness probe (if defined) and identify the reason a restart was necessary.

The following Pod observes the status `ErrImagePull` and makes 0/1 containers available to incoming traffic. In short, this Pod has a problem:

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
misbehaving-pod	0/1	ErrImagePull	0	2s

After working with Kubernetes for a while, you'll automatically recognize common error conditions. [Table 21-1](#) lists some of those error statuses and explains how to fix them.

Table 21-1. Common Pod error statuses

Status	Root cause	Potential fix
<code>ImagePullBackOff</code> or <code>ErrImagePull</code>	Image could not be pulled from registry.	Check correct image name, check that image name exists in registry, verify network access from node to registry, ensure proper authentication.
<code>CrashLoopBackOff</code>	Application or command run in container crashes.	Check command executed in container, ensure that image can properly execute (e.g., by creating a container with Docker).
<code>CreateContainerConfigError</code>	ConfigMap or Secret referenced by container cannot be found.	Check correct name of the configuration object, verify the existence of the configuration object in the namespace.

Inspecting Events

You might not encounter any of those error statuses. But there's still a chance of the Pod having a configuration issue. You can retrieve detailed information about the Pod using the `kubectl describe pod` command to inspect its events.

The following output belongs to a Pod that tries to mount a Secret that doesn't exist. Instead of rendering a specific error message, the Pod gets stuck with the status `ContainerCreating`:

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
secret-pod	0/1	ContainerCreating	0	4m57s

```
$ kubectl describe pod secret-pod
```

```
...
```

```
Events:
```

Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	Scheduled	<unknown>	default-scheduler	Successfully \ assigned \ default/secret-pod to minikube
Warning	FailedMount	3m15s	kubelet, minikube	Unable to attach mount volumes: \ unmounted \ volumes=[mysecret] unattached volume [default-token-bdmysecret]: timed out waiting for the condition
Warning	FailedMount	68s (x10 over 5m18s)	kubelet, minikube	MountVolume.SetUp failed for volume "mysecret" : secret "mysecret" not found
Warning	FailedMount	61s	kubelet, minikube	Unable to attach mount volumes: \ unmounted volume [mysecret], \ unattached \ volumes=[mysecret default-token-bdmysecret] : timed out \ waiting for the condition

Another helpful command is `kubectl get events`. The output of the command lists the events across all Pods for a given namespace. You can use additional command-line options to further filter and sort events:

```
$ kubectl get events
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
3m14s	Warning	BackOff	pod/custom-cmd	Back-off \ restarting \ failed container
2s	Warning	FailedNeedsStart	cronjob/google-ping	Cannot determine if job needs to be started

```
be started: to
many missed st
time (> 100).
or decrease \
.spec. \
startingDeadli
Seconds or che
clock skew
```

Sometimes troubleshooting won't be enough. You may have to dig into the application runtime behavior and configuration in the container.

Using Port Forwarding

In production environments, you'll operate an application in multiple Pods controlled by a ReplicaSet. It's not unusual that one of those replicas experiences a runtime issue. Instead of troubleshooting the problematic Pod from a temporary Pod from within the cluster, you can also forward the traffic to a Pod through a tunneled HTTP connection. This is where the `port-forward` [command](#) comes into play.

Let's demonstrate the behavior. The following command creates a new Deployment running NGINX in three replicas:

```
$ kubectl create deployment nginx --image=nginx:1.24.0 --replicas=3 --port=80
deployment.apps/nginx created
```

The resulting Pod will have unique names derived from the name of the Deployment. Say the Pod `nginx-595dff4799-ph4js` has an issue you want to troubleshoot:

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-595dff4799-pfgdg	1/1	Running	0	6m25s
nginx-595dff4799-ph4js	1/1	Running	0	6m25s
nginx-595dff4799-s76s8	1/1	Running	0	6m25s

The `port-forward` command forwards HTTP connections from a local port to a port exposed by a Pod. This exemplary command forwards port 2500 on your local machine to the container port 80 running in the Pod `nginx-595dff4799-ph4js`:

```
$ kubectl port-forward nginx-595dff4799-ph4js 2500:80
Forwarding from 127.0.0.1:2500 -> 80
Forwarding from [::1]:2500 -> 80
```

The `port-forward` command runs in the foreground and doesn't return to the shell prompt, requiring you to either open another terminal to perform calls to the Pod via the forwarded port, or use your shell's job control features—pressing Ctrl+Z to suspend it followed by `bg` to run it in the background, or appending `&` when starting the command (e.g., `kubectl port-forward mypod 8080:80 &`) to immediately background the process while keeping the port forwarding active.

The following command simply checks if the Pod is accessible from your local machine using `curl`:

```
$ curl -Is localhost:2500 | head -n 1
HTTP/1.1 200 OK
```

The HTTP response code 200 clearly shows that we can access the Pod from outside of the cluster. The `port-forward` command is not meant to run for a long time. Its primary purpose is for testing or troubleshooting a Pod without having to expose it with the help of a Service.

Troubleshooting Containers

You can interact with the container for a deep-dive into the application's runtime environment. The next sections will discuss how to inspect logs, open an interactive shell to a container, and debug containers that do not provide a shell.

TARGETING A SPECIFIC CONTAINER

The commands described in the following sections apply to init and sidecar containers as well. Use the `-c` or `--container` command-line flag to target a specific container if you are running more than a single one.

Inspecting Logs

When troubleshooting a Pod, you can retrieve the next level of details by downloading and inspecting its logs. You may or may not find additional information that points to the root cause of a misbehaving Pod, but it's definitely worth a look. The YAML manifest shown in [Example 21-1](#) defines a Pod running a shell command.

Example 21-1. A Pod running a failing shell command

```
apiVersion: v1
kind: Pod
```

```

metadata:
  name: incorrect-cmd-pod
spec:
  containers:
  - name: test-container
    image: busybox:1.36.1
    command: [ "/bin/sh", "-c", "unknown" ]

```

After creating the object, the Pod fails with the status

CrashLoopBackOff. Running the `logs` command reveals that the command run in the container has an issue:

```

$ kubectl create -f crash-loop-backoff.yaml
pod/incorrect-cmd-pod created
$ kubectl get pods incorrect-cmd-pod

```

NAME	READY	STATUS	RESTARTS	AGE
incorrect-cmd-pod	0/1	CrashLoopBackOff	5	3m20s

```

$ kubectl logs incorrect-cmd-pod
/bin/sh: unknown: not found

```

The `logs` command provides two helpful options. The option `-f` streams the logs, meaning you'll see new log entries as they're being produced in real time. The option `--previous` gets the logs from the previous instantiation of a container, which is helpful if the container has been restarted.

Opening an Interactive Shell

If any of the previous commands don't point you to the root cause of the failing Pod, it's time to open an interactive shell to a container.

Application developers will know best what behavior to expect from the application at runtime. Inspect the running processes by using the Unix or Windows utility tools, depending on the image run in the container.

Say you encounter a situation where a Pod seems to work properly on the surface, as shown in [Example 21-2](#).

Example 21-2. A Pod periodically writing the current date to a file

```

apiVersion: v1
kind: Pod
metadata:
  name: failing-pod
spec:
  containers:
  - args:
    - /bin/sh
    - -c

```

```
- while true; do echo $(date) >> ~/tmp/curr-date.txt; sleep \
5; done;
image: busybox:1.36.1
name: failing-pod
```

After creating the Pod, you check the status. It says `Running`; however, when making a request to the application, the endpoint reports an error. Next, you check the logs. The log output renders an error message that points to a nonexistent directory. Apparently, the directory that the application needs hasn't been set up correctly:

```
$ kubectl create -f failing-pod.yaml
pod/failing-pod created
$ kubectl get pods failing-pod
NAME          READY   STATUS    RESTARTS   AGE
failing-pod   1/1     Running   0           5s
$ kubectl logs failing-pod
/bin/sh: can't create /root/tmp/curr-date.txt: nonexistent directory
```

The `exec` command opens an interactive shell to troubleshoot in a hands-on fashion. In the following code, we're using the Unix tools `mkdir`, `cd`, and `ls` inside of the running container to fix the problem. Obviously, the better mitigation strategy is to create the directory from the application or provide an instruction in the Dockerfile:

```
$ kubectl exec failing-pod -it -- /bin/sh
# mkdir -p ~/tmp
# cd ~/tmp
# ls -l
total 4
-rw-r--r-- 1 root root 112 May 9 23:52 curr-date.txt
```

Interacting with a Distroless Container

Some images run in containers are designed to be very minimal for security reasons. For example, the [Google distroless](#) images don't have any Unix utility tools preinstalled. You can't even open a shell to a container, as it doesn't come with a shell.

INCORPORATING SECURITY BEST PRACTICES FOR CONTAINER IMAGES

Shipping container images with accessible shells and running with the `root` user is commonly discouraged, as these aspects can be used as potential attack vectors. Check out the [CKS certification](#) to learn more about security concerns in Kubernetes.

One of Google's distroless images is `k8s.gcr.io/pause:3.1`, shown in [Example 21-3](#).

Example 21-3. Running a distroless image

```
apiVersion: v1
kind: Pod
metadata:
  name: minimal-pod
spec:
  containers:
  - image: k8s.gcr.io/pause:3.1
    name: pause
```

As you can see in the following `exec` command, the image doesn't provide a shell:

```
$ kubectl create -f minimal-pod.yaml
pod/minimal-pod created
$ kubectl get pods minimal-pod
NAME          READY   STATUS    RESTARTS   AGE
minimal-pod   1/1     Running   0           8s
$ kubectl exec minimal-pod -it -- /bin/sh
OCI runtime exec failed: exec failed: container_linux.go:349: starting \
container process caused "exec: \"/bin/sh\": stat /bin/sh: no such file \
or directory": unknown
command terminated with exit code 126
```

Kubernetes offers the concept of *ephemeral containers*. Those containers are meant to be disposable and have no resilience features like probes. You can deploy an ephemeral container for troubleshooting minimal containers that would usually not allow the use of the `exec` command.

The `debug` command can inject an ephemeral container to a running Pod for debugging purposes. The following command adds the ephemeral container running the image `busybox` to the Pod named `minimal-pod` and opens an interactive shell for it:

```
$ kubectl debug -it minimal-pod --image=busybox:1.37.0
Defaulting debug container name to debugger-jf98g.
If you don't see a command prompt, try pressing enter.
/ # pwd
/
/ # exit
Session ended, resume using 'kubectl alpha attach minimal-pod -c \
debugger-jf98g -i -t' command when the pod is running
```


Troubleshooting Services and Networking

A Service provides a unified network interface for Pods. For full coverage on networking aspects in Kubernetes, see [Chapter 17](#). Here, I want to point out troubleshooting techniques for this primitive.

Service and networking problems in Kubernetes are among the most challenging issues to diagnose because they involve multiple layers: Pod networking, service discovery, DNS resolution, network policies, and ingress controllers. These issues typically manifest as connection time-outs, refused connections, or intermittent failures that can cripple application functionality.

Diagnosing Service-to-Pod Label Selection

In case you can't reach the Pods that should map to the Service, start by ensuring that the label selector matches with the assigned labels of the Pods. You can query the information by describing the Service and then render the labels of the available Pods with the option `--show-labels`. The following example does not have matching labels and therefore wouldn't apply to any of the Pods running in the namespace:

```
$ kubectl describe service myservice
...
Selector:          app=myapp
...
$ kubectl get pods --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
myapp-68bf896d89-qfhlv	1/1	Running	0	7m39s	app=hello
myapp-68bf896d89-tzt55	1/1	Running	0	7m37s	app=world

Diagnosing Service-to-Pod Port Mapping

Services must correctly map ports between the Service definition and the selected Pod's containers. Check if the port mapping from the target port of the Service to the container port of the Pod is configured correctly. Both ports need to match, or the network traffic wouldn't be routed properly:

```
$ kubectl get service myapp -o yaml | grep targetPort:
  targetPort: 80
$ kubectl get pods myapp-68bf896d89-qfhlv -o yaml | grep containerPort:
- containerPort: 80
```

Inspecting the Service's Endpoints

A straightforward way to check if label selection and port mapping are set up properly is to inspect the Service's endpoints. Service endpoints are API objects that represent the network addresses (IP addresses and ports) of the Pods backing a Service.

Use the `get endpoints` command to render a Service's endpoints. Ensure that the output lists the number of Pod IP addresses and container ports expected to be selected by the Service:

```
$ kubectl get endpoints myservice
```

NAME	ENDPOINTS	AGE
myservice	172.17.0.5:80,172.17.0.6:80	9m31s

If the output of the command does not render any endpoints, then you know that either label selection or port mapping has not been configured correctly in the Service.

Verifying Accessibility Scope

Different service types (`ClusterIP`, `NodePort`, `LoadBalancer`) have different accessibility scopes. By default, the Service type is `ClusterIP`, which means that a Pod can be reached through the Service only if queried from inside of the cluster. First, check the Service type:

```
$ kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
myservice	ClusterIP	10.99.155.165	<none>	80/TCP	15m
...					

If you think that `ClusterIP` is the proper type you wanted to assign, open an interactive shell from a temporary Pod inside the cluster and run a `curl` or `wget` command:

```
$ kubectl run tmp --image=busybox:1.37.0 -it --rm -- wget 10.99.155.165:80
```

DNS Resolution Problems

Instead of using the cluster-internal IP address to access a `ClusterIP` Service, you will likely want to use the Service's DNS name instead:

```
$ kubectl run tmp --image=busybox:1.37.0 -it --rm -- wget myservice:80
```

Before jumping to any conclusions, remember that Services in different namespaces require fully qualified domain names (FQDN). The more bulletproof option to check accessibility to a Service is to include the namespace:

```
$ kubectl run tmp --image=busybox:1.37.0 -it --rm -- wget \
myservice.default.svc.cluster.local:80
```

DNS issues can prevent Pods from resolving service names, causing application failures. It's possible that the CoreDNS Pods are currently not running. You can check by executing the following command:

```
$ kubectl get pods -n kube-system -l k8s-app=kube-dns
```

To scan the CoreDNS logs for error messages, run the following command:

```
$ kubectl logs -n kube-system -l k8s-app=kube-dns --tail=50
```

CoreDNS logs reveal DNS-related problems through various error patterns that help identify the root cause of service discovery failures. Common issues include upstream connectivity problems where CoreDNS cannot reach external DNS servers, resolution failures for Services that don't exist or are incorrectly referenced, communication breakdowns with the Kubernetes API server preventing Service discovery, configuration problems causing routing loops or processing errors, and performance issues from either excessive query loads or insufficient resources. By recognizing these patterns in the logs, you can quickly determine whether the problem lies in network connectivity, configuration, or resource allocation.

Network Policy Restrictions

Network policies in Kubernetes act as a firewall at the Pod level, controlling which Pods can communicate with each other and what external traffic is allowed. When network policies are implemented, they follow a “default deny” model—once any network policy selects a Pod, that Pod becomes isolated and can only receive traffic explicitly allowed by policies.

This security feature, while essential for production environments, frequently causes mysterious connectivity issues where applications that previously worked suddenly experience timeouts or connection refused errors. Common symptoms include Pods being unable to reach services

they depend on, health checks failing, cross-namespace communication breaking, or external traffic being blocked unexpectedly.

The challenge with troubleshooting network policies lies in their implicit nature—there’s no immediate error message indicating that a network policy is blocking traffic; instead, connections simply fail silently.

Debugging requires systematically checking if network policies exist in the namespace, understanding which Pods they select through their `podSelector`, examining the ingress and egress rules to see what traffic is permitted, and testing connectivity from different source Pods to identify exactly where the policy enforcement is occurring.

The following command checks if any network policies exist in the namespace `production`:

```
$ kubectl get networkpolicies -n production
```

To see which Pods are affected by network policies, render its details. The following command assumes that we found the network policy `api-policy` to exist:

```
$ kubectl describe networkpolicy api-policy -n production
```

Lastly, try to test connectivity from one Pod to another one either directly or through a Service in the same namespace. The following command runs a `curl` command against a Service named `backend-service`:

```
$ kubectl exec -it frontend-pod -n production -- curl http://backend-service
```

If this times out, but the backend is running, it’s fair to assume that the network policy comes into play here.

Inspecting Resource Metrics

Deploying software to a Kubernetes cluster is only the start of operating an application long-term. Developers need to understand their applications’ resource consumption patterns and behaviors, with the goal of providing a scalable and reliable service.

In the Kubernetes world, monitoring tools like [Prometheus](#) and [Datadog](#) help with collecting, processing, and visualizing information over time. The exam does not expect you to be familiar with third-party monitoring,

logging, tracing, and aggregation tools; however, it is helpful to have a basic understanding of the underlying Kubernetes infrastructure responsible for collecting usage metrics. The following are examples of typical metrics:

- Number of nodes in the cluster
- Health status of nodes
- Node performance metrics such as CPU, memory, disk space
- Pod-level performance metrics such as CPU and memory consumption

This responsibility falls to the [Metrics Server](#), a cluster-wide aggregator of resource usage data. As shown in [Figure 21-1](#), kubelets running on nodes collect metrics and send them to the Metrics Server.

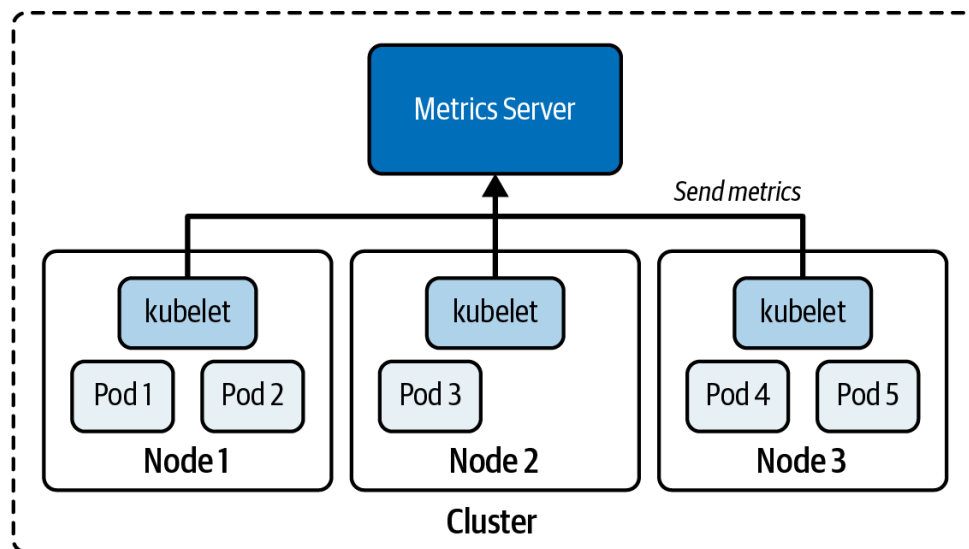


Figure 21-1. Data collection for the Metrics Server

The Metrics Server stores data in memory and does not persist data over time. If you are looking for a solution that keeps historical data, then you need to look into commercial or self-hosted options. Refer to the documentation for more information on its [installation process](#).

If you're using minikube as your practice environment, [enabling the Metrics Server add-on](#) is straightforward using the following command:

```
$ minikube addons enable metrics-server
```

```
The 'metrics-server' addon is enabled
```

You can now query for metrics of cluster nodes and Pods with the `top` command:

```
$ kubectl top nodes
```

NAME	CPU(cores)	CPU%	MEMORY(bytes)	MEMORY%
minikube	283m	14%	1262Mi	32%

```
$ kubectl top pod frontend
```

NAME	CPU(cores)	MEMORY(bytes)
frontend	0m	2Mi

It takes a couple of minutes after the installation of the Metrics Server before it has gathered information about resource consumption. Rerun the `kubectl top` command if you receive the error message `error: Metrics API not available`.

Summary

This chapter discussed strategies for approaching failed or misbehaving Pods. The main goal is to diagnose the root cause of a failure and then fix it by taking the right action. Troubleshooting Pods doesn't have to be hard. With the right `kubectl` commands in your tool belt, you can rule out root causes one by one to get a clearer picture.

Effective Kubernetes network troubleshooting follows a systematic approach that begins with verifying the fundamentals, ensuring Pods are running, Services have healthy endpoints, and DNS resolution is functioning, before moving to more complex investigations. Most importantly, understand the capabilities and limitations of each Service type (`ClusterIP` for internal traffic, `NodePort` for node-level access, and `LoadBalancer` for external exposure) to set appropriate expectations and choose the right troubleshooting approach for each scenario.

The Kubernetes ecosystem provides a lot of options for collecting and processing metrics of your cluster over time. Among those options are monitoring tools like Prometheus and Datadog. Many of those tools use the Metrics Server as the source of truth for those metrics. We also briefly touched on the installation process and the `kubectl top` command for retrieving metrics from the command line.

Exam Essentials

Know how to troubleshoot applications

Applications running in a Pod can easily break due to misconfiguration. Think of possible scenarios that can occur and try to model them proactively to represent a failure situation. Then using the commands `get`, `logs`, and `exec`, get to the bottom of the issue and fix it. Try to dream up obscure scenarios to become more comfortable with finding and fixing application issues for different resource types. Refer to the [Kubernetes documentation](#) to learn more about debugging other Kubernetes resource types.

Make accessing container logs your daily bread and butter

Accessing container logs is straightforward. Simply use the `logs` command. Practice the use of all relevant command-line options. The option `-c` targets a specific container. The option does not have to be used explicitly for single-container Pods. The option `-f` follows the log entries if you want to see live processing in an application. The `-p` option can be used for accessing logs if the container needed to be restarted, but you still want to take a look at the previous container logs.

Know how to diagnose networking issues

Follow a systematic approach: first verify basic connectivity and DNS, then check service endpoints and selectors, examine port configurations, investigate any network policies, and finally test the complete traffic path from source to destination. Always remember that the exam environment may have preconfigured network policies or specific networking plug-ins that affect troubleshooting strategies.

Learn how to retrieve and interpret resource metrics

Monitoring a Kubernetes cluster is an important aspect of successfully operating in a real-world environment. You should read up on commercial monitoring products and which data the Metrics Server can collect. You can assume that the exam environment provides you with an installation of the Metrics Server. Learn how to use the `kubectl top` command to render Pod and node resource metrics and how to interpret them.

Sample Exercises

Solutions to these exercises are available in [Appendix A](#).

1. In this exercise, you will practice your troubleshooting skills by inspecting a misconfigured Pod. Navigate to the directory `app-a/ch21/troubleshooting-pod` of the checked-out GitHub repository [bmuschko/cka-study-guide](#). Create a new Pod from the YAML manifest in the file `setup.yaml`. Check the Pod's status. Do you see any issues? Render the logs of the running container and identify an issue. Shell into the container. Can you verify the issue based on the rendered log message? Suggest solutions that can fix the root cause of the issue.
2. Kate is a developer in charge of implementing a web-based application stack. She is not familiar with Kubernetes, and asked if you could

help out. The relevant objects have been created; however, connection to the application cannot be established from within the cluster. Help Kate with fixing the configuration of her YAML manifests.

Navigate to the directory *app-a/ch21/troubleshooting-service* of the checked-out GitHub repository [bmuschko/cka-study-guide](#). Create the objects from the YAML manifest *setup.yaml*. Inspect the objects in the namespace `y72`.

Create a temporary Pod using the image `busybox:1.36.1` in the namespace `y72`. The container command should make a `wget` call to the Service `web-app`. The `wget` call will not be able to establish a successful connection to the Service.

Identify the root cause for the connection issue and fix it. Verify the correct behavior by repeating the previous step. The `wget` call should return a successful response.

3. You will inspect the metrics collected by the Metrics Server. Navigate to the directory *app-a/ch21/stress-test* of the checked-out GitHub repository [bmuschko/cka-study-guide](#). The current directory contains the YAML manifests for three Pods, *stress-1-pod.yaml*, *stress-2-pod.yaml*, and *stress-3-pod.yaml*. Inspect those manifest files.

Create the namespace `stress-test` and the Pods inside of the namespace.

Use the data available through the Metrics Server to identify which of the Pods consumes the most memory.

Prerequisite: You will need to install the Metrics Server if you want to be able to inspect actual resource metrics. You can find [installation instructions](#) on the project's GitHub page.